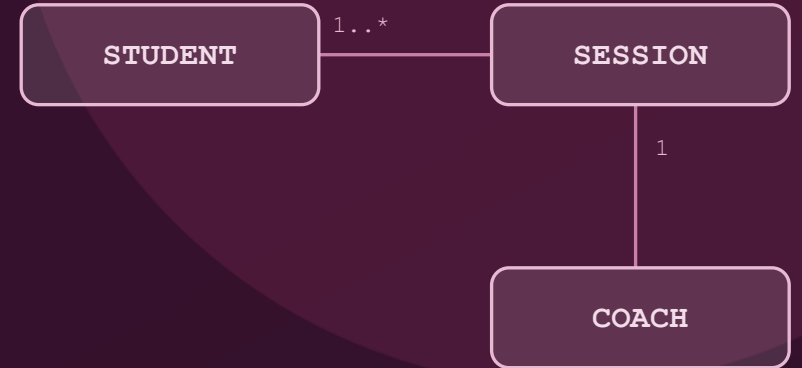


Coaching System for Students

A relational database design for goal tracking, coaching sessions, resources and student progress.



What This Deck Covers



System Request

Five functional modules the platform must deliver.



Business Rules

33 relationship rules across students, coaches and records.



Class Diagram

UML view of all entities and their associations.



Logical Diagram

Table-level design with keys and sample rows.



10 SQL Operations

DDL and DML statements that exercise the schema.



View, Procedure, Trigger

Three programmable objects built on the model.

Business Need

Many students struggle to reach their academic goals, manage time effectively, and receive timely guidance for personal and academic development. The problem is sharpest for students preparing for the university entrance exam.

Traditional advising is limited by time constraints, so consistent and personalized support is hard to sustain. The Student Coaching System closes this gap with a centralized platform for tracking progress and setting personal and academic goals.

Students stay on track with their goals and improve their academic outcomes, while coaches manage and monitor their students far more efficiently.



Problem

Advising time is scarce; support is inconsistent and generic.



Goal

Personalized, continuous coaching for exam-focused students.



Solution

One platform for goals, sessions, resources and progress.

System Request: Five Modules



1. Student Management

Registration, profile updates and tracking of learning activity: enrolled packages, completed sessions, progress toward goals.



2. Coach Management

Coach profiles with specialization, availability, scheduled sessions, feedback received and assigned tasks.



3. Session Scheduling

Students book, reschedule or cancel sessions with coaches and track the completion status of each one.



4. Resource Materials

Coaches upload, assign and manage learning resources such as guides and videos, per package or per session.



5. Package Management

Learning packages with price and session count, student enrollment, and links to specific goals and resources.

Together these modules cover the full coaching journey: enroll, plan, meet, learn, track.

Student Relationships

Student - Package

- A student can enroll in many packages.
- A package can have many students.

Student - Assignment

- A student completes many assignments.
- An assignment belongs to one student.

Student - Feedback

- A student receives many feedback entries.
- Feedback is given to only one student.

Student - Progress Report

- One progress report per package.
- A report holds one student's activity.

Student - Favorite Coach

- One favorite coach at a time.
- A coach can be favorited by many.

Student - Goal

- A student sets many goals.
- A goal belongs to one student.

Student - Action Plan

- A student has multiple action plans.
- Each plan is tied to that student's goals.

Student - Session

- A student attends many sessions.
- A session can host multiple students.

Coach and Planning

Action Plan - Goal

- A plan can cover multiple goals.
- A goal belongs to one action plan.

Coach - Action Plan

- A coach oversees many action plans.
- A plan is guided by only one coach.

Coach - Schedule

- A coach has one schedule.
- It holds all sessions and availability.

Coach - Availability

- Availability is defined by time slots.
- Each slot belongs to one coach.

Coach - Session

- A coach conducts many sessions.
- A session has only one coach.

Coach - Favorite

- A coach cannot favorite themselves.
- Coaches are favorited by students only.

Coach - Feedback

- A coach gives many feedback entries.
- Each entry targets one student.

Feedback - Session

- Feedback is tied to one session.
- One entry per student in a session.

Notifications and Resources

Notification - Goal

- A notification comes from one goal.
- A goal generates one notification.

Notification - Action Plan

- A notification maps to one plan.
- A plan generates one notification.

Notification - Feedback

- A notification maps to one entry.
- Feedback generates one notification.

Notification - Participant

- Linked to one communication participant.
- A participant generates many.

Notification - Progress Report

- Linked to one progress report.
- A report generates one notification.

Resource - Package

- A resource belongs to one package.
- A package has multiple resources.

Resource - Coach

- A resource is assigned by one coach.
- A coach assigns many resources.

Reward - Student

- A student earns many rewards.
- A reward can go to many students.

Enrollment and Records

Comm. Log - Participant

- A log entry links to one participant.
- A participant has many interactions.

Participant - Coach

- A participant can be a coach.
- A coach has many log entries.

Participant - Student

- A participant can be a student.
- A student has many log entries.

Certificate - Student

- A student earns many certificates.
- A certificate goes to many students.

Course - Student

- A course enrolls many students.
- A student joins multiple courses.

Certificate - Package

- A certificate is linked to one package.
- A package has many certificates.

Payment - Student

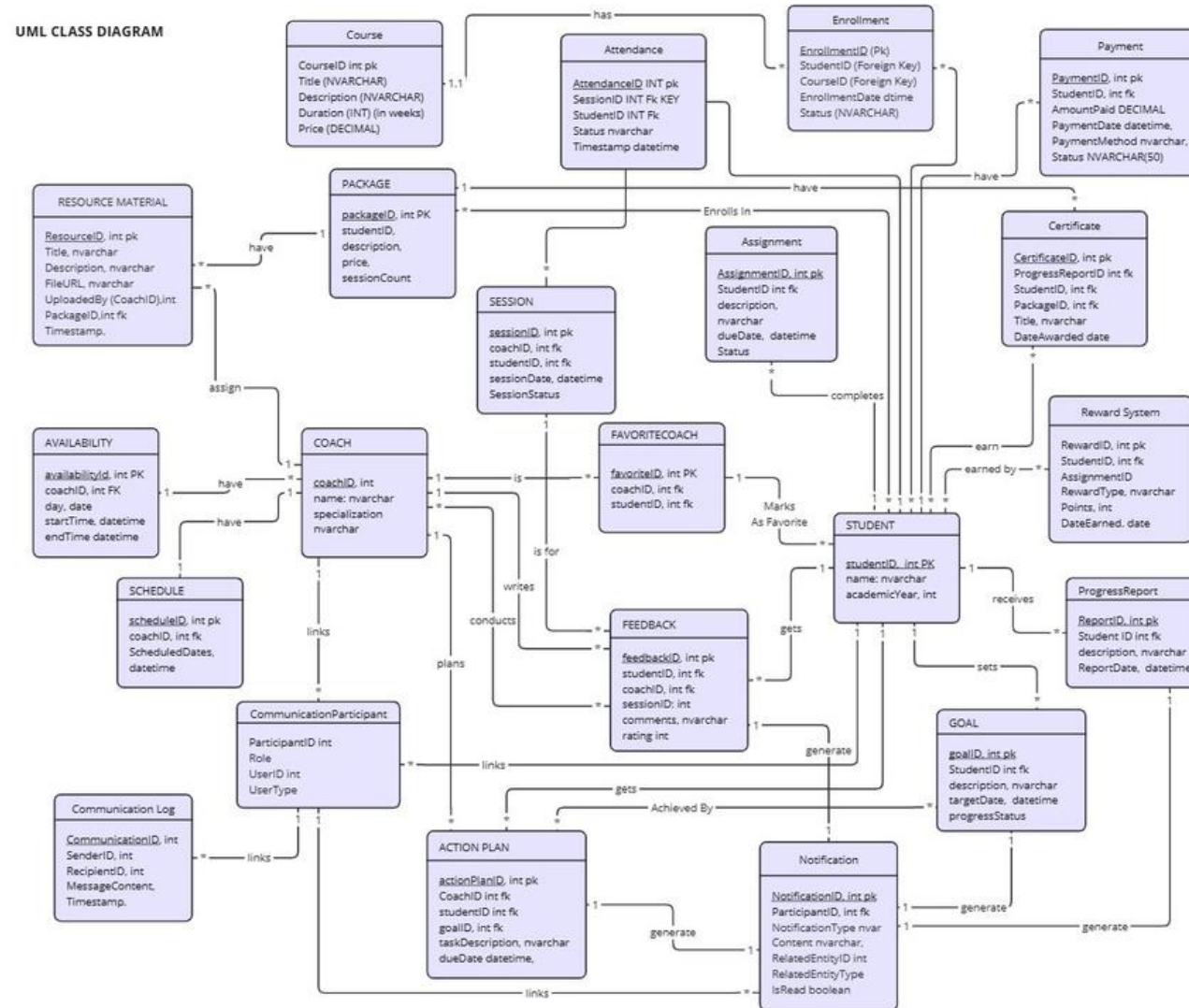
- A payment belongs to one student.
- A student makes multiple payments.

Attendance - Session

- Attendance is recorded per session.
- A session has records for many students.

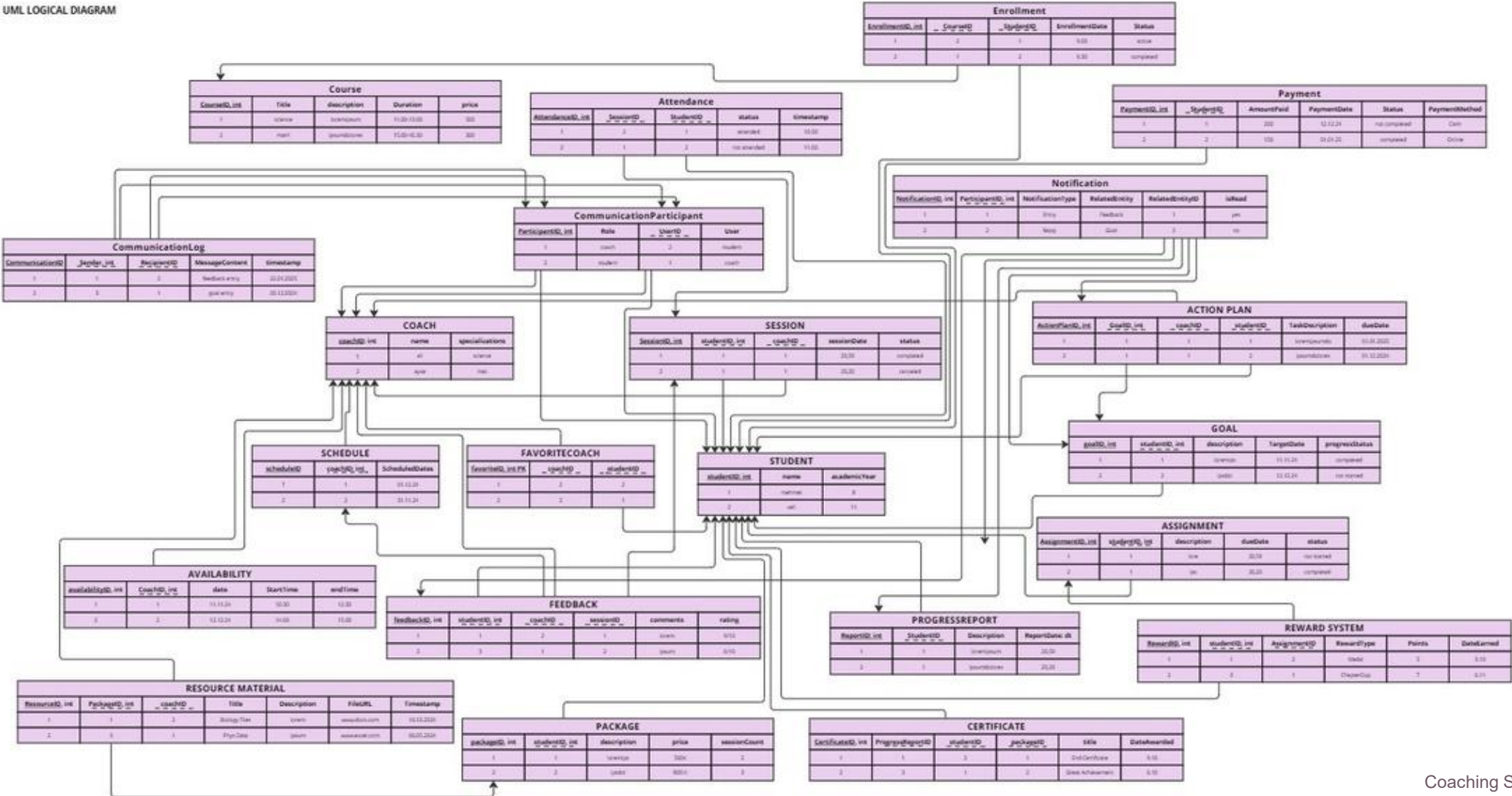
Enrollment - Course

- An enrollment links to one course.
- A course has many enrollments.



UML Logical Diagram

UML LOGICAL DIAGRAM



10 SQL Operations

1 CREATE

2 ALTER

3 INSERT

4 UPDATE

5 DELETE

6 SELECT with GROUP BY

7 SELECT with ORDER BY

8 SELECT with WHERE

9 SELECT with INNER JOIN

10 SELECT with WHERE, GROUP BY, ORDER BY

SQL Operations

1 CREATE

Create a table to store student goals.

```
CREATE TABLE Goal (  
    GoalID INT IDENTITY(1,1) PRIMARY KEY,  
    StudentID INT,  
    Description NVARCHAR(255),  
    TargetDate DATE,  
    ProgressStatus NVARCHAR(255),  
    FOREIGN KEY (StudentID)  
        REFERENCES Student(StudentID)  
);
```

2-3 ALTER + INSERT

Rebuild the schedule date column as datetime, then insert a new student.

```
ALTER TABLE Schedule  
    DROP COLUMN AvailableDate;  
ALTER TABLE Schedule  
    ADD ScheduledDates DATETIME;  
  
INSERT INTO Student (Name, AcademicYear)  
VALUES ('Tuna', 4);
```

SQL Operations

4 UPDATE

Mark a goal as completed.

```
UPDATE Goal
SET ProgressStatus = 'Completed'
WHERE GoalID = 1
    AND StudentID = 2;
```

5 DELETE

Delete a session record by its ID.

```
DELETE FROM Session
WHERE SessionID = 3;
```

SQL Operations

6 SELECT with GROUP BY

Count the students enrolled in each package.

```
SELECT PackageID,  
       COUNT(StudentID) AS TotalStudents  
FROM Package  
GROUP BY PackageID;
```

7 SELECT with ORDER BY

List feedback ratings from highest to lowest.

```
SELECT StudentID, FeedbackID, Rating  
FROM Feedback  
ORDER BY Rating DESC;
```

SQL Operations

8 SELECT with WHERE

Find all feedback given to a student by their coach.

```
SELECT Comments
FROM Feedback
WHERE StudentID = 2 AND CoachID = 2;
```

9 SELECT with INNER JOIN

Fetch students and their coaches for attended sessions.

```
SELECT s.StudentID,
       s.Name AS StudentName,
       c.Name AS CoachName,
       se.SessionID
FROM Student s
INNER JOIN Session se
    ON s.StudentID = se.StudentID
INNER JOIN Coach c
    ON se.CoachID = c.CoachID;
```

WHERE + GROUP BY + ORDER BY

Find the average feedback rating per student and rank them from best to worst.

Why it matters

- Combines filtering, aggregation and sorting in one query.
- Powers the coach dashboard's student ranking.
- AVG() ignores sessions without a rating thanks to WHERE.

```
SELECT StudentID,  
       AVG(Rating) AS AvgRating  
FROM Feedback  
WHERE Rating > 0  
GROUP BY StudentID  
ORDER BY AvgRating DESC;
```


View: StudentProgressView

A reusable report that joins students, progress reports and feedback into one queryable table of names, activities and scores.



*SELECT * FROM StudentProgressView gives coaches the whole picture in one line.*

```
GO
CREATE VIEW StudentProgressView AS
SELECT
    s.StudentID,
    s.Name          AS StudentName,
    pr.ReportID     AS ProgressID,
    pr.Description  AS ActivityName,
    f.Rating        AS Score,
    pr.ReportDate   AS CompletionDate
FROM Student s
JOIN ProgressReport pr
    ON s.StudentID = pr.StudentID
LEFT JOIN Feedback f
    ON s.StudentID = f.StudentID
    AND pr.ReportID = f.SessionID;
```

Stored Procedure: AddGoal

Inserts a new goal for a student with a single call. New goals always start in 'Pending' status, so the rule lives in the database, not in the app.



```
EXEC AddGoal 2, 'Finish calculus set',  
'2025-01-10', '2025-02-01';
```

```
GO  
CREATE PROCEDURE AddGoal  
    @student_id INT,  
    @description NVARCHAR(255),  
    @start_date DATE,  
    @end_date DATE  
AS  
BEGIN  
    INSERT INTO Goal  
        (StudentID, Description,  
         TargetDate, ProgressStatus)  
    VALUES (@student_id, @description,  
            @end_date, 'Pending');  
END;
```

Trigger: NotifyStudentOnFeedback

Fires after every INSERT on Feedback and writes a matching row into Notification, so students are alerted the moment a coach leaves feedback.



No application code needed: the database keeps notifications consistent by itself.

```
GO
CREATE TRIGGER NotifyStudentOnFeedback
ON Feedback AFTER INSERT AS
BEGIN
    SET NOCOUNT ON;
    INSERT INTO Notification
        (ParticipantID, NotificationType,
         Content, RelatedEntityID,
         RelatedEntityType, IsRead)
    SELECT
        i.StudentID,
        'Feedback',
        CONCAT('New feedback received: ',
              i.Comments),
        i.FeedbackID, 'Feedback', 0
    FROM INSERTED i;
END;
```

THANK YOU!

Coaching System for Students · Cagla Eraslan · CET 321

